

MovieLens 大作业实验报告

一、传统偏好发现

指标选择

本题的关键在于如何度量每个职业对每个电影的偏好程度。观察demo中的指标设定，我们可以这样定义一个通用的偏好程度的度量：

$$preference = difference \times representativeness$$

其中 `difference` 指与其它群体或与整体的差异程度，`representativeness` 则表示观看该类电影的属于特定群体的人在该群体中的代表性，即这些人的偏好能否代表其所属的整个群体的偏好。

在职业偏好分析的上下文中，对于职业 `i` 和电影 `j`，我们定义：

$$difference_{i,j} = mean_{i,j} - mean_{all,j}$$
$$representativeness_{i,j} = \frac{WatchingNumber_{i,j}}{WatchingNumber_{all,j}} + 2 \times \frac{WatchingNumber_{i,j}}{OccupationNumber_i}$$

这里着重讨论 `representativeness` 的定义。如何刻画观影人评分的代表性？我认为主要需要考虑：

1) 观看电影 `j` 的人中从事职业 `i` 的人比例

该比例越高，则可认为电影 `j` 可能更受从事职业 `i` 的人欢迎，即其可能更加偏好电影 `j`

2) 从事职业 `i` 的人中观看了电影 `j` 的比例

但只考虑1) 也有着明显的问题，如果电影 `j` 非常小众（或者说非常不受欢迎），只有极少数人观看了电影，而这其中从事职业 `i` 的人碰巧比例较高。尽管观看电影 `j` 的人中从事职业 `i` 的人比例较高，但是这些人在职业 `i` 中只是极少数，这些人对电影的评分不能代表职业 `i` 对电影 `j` 的偏好程度。因此我们引入2)，并给予其较高的权重。

1. 衡量偏好程度的指标与偏好展示

1) 指标的计算

统计每种职业的人数

Each occupation's number:

	occupation	occupation total number
0	K-12 student	195
1	academic/educator	528
2	artist	267
3	clerical/admin	173
4	college/grad student	759
5	customer service	112
6	doctor/health care	236
7	executive/managerial	679
8	farmer	17
9	homemaker	92
10	lawyer	129
11	other or not specified	711
12	programmer	388
13	retired	142
14	sales/marketing	302
15	scientist	144
16	self-employed	241
17	technician/engineer	502
18	tradesman/craftsman	70
19	unemployed	72
20	writer	281

计算每个职业的观影者对每部电影的评分以及每个职业观看每部电影的人数

Example: doctor/health care:

	movie_id	rating	number
0	1	4.394737	76
1	2	3.107143	28
2	3	2.714286	21
3	6	4.068966	29
4	7	3.214286	14

计算所有观影者对电影的平均评分并合并数据

all_ratings:

	index	movie_id	rating	number	occupation	occupation total number	overall_rating
0	0	1	3.904255	94	K-12 student	195	4.151042
1	0	1	4.229299	157	academic/educator	528	4.151042
2	0	1	4.000000	86	artist	267	4.151042
3	0	1	4.492537	67	clerical/admin	173	4.151042
4	0	1	4.040404	297	college/grad student	759	4.151042

计算差异与观影的总人数

all_ratings:

	index	movie_id	rating	number	occupation	occupation total number	overall_rating	diff_from_avg	count
0	0	1	3.904255	94	K-12 student	195	4.151042	-0.246786	2077
1	0	1	4.229299	157	academic/educator	528	4.151042	0.078258	2077
2	0	1	4.000000	86	artist	267	4.151042	-0.151042	2077
3	0	1	4.492537	67	clerical/admin	173	4.151042	0.341496	2077
4	0	1	4.040404	297	college/grad student	759	4.151042	-0.110638	2077

计算 preference

```
w1 = 1
w2 = 2
all_ratings['score'] = all_ratings['diff_from_avg'] *
    (w1 * all_ratings['number'] / all_ratings['count'] +
    w2 * all_ratings['number'] / all_ratings['occupation total number'])
```

Example: doctor/health care:

	number	occupation total number	diff_from_avg	count	score
movie_id					
1	76	236	0.243695	2077	0.165873
2	28	236	-0.065700	701	-0.018214
3	21	236	-0.306814	478	-0.068082
6	29	236	0.176449	940	0.048808
7	14	236	-0.164260	458	-0.024509
...	...	...	...	...	...
3916	19	236	0.223229	437	0.045649
3927	12	236	0.264032	348	0.035955
3948	30	236	-0.059601	862	-0.017227
3949	8	236	0.056045	304	0.005275
3952	19	236	0.319830	388	0.067160

2) 不同职业观影者的偏好展示

筛选出每个职业前10名的电影，分别作为该职业偏好的电影

K-12 student 最偏好的10部电影:

	score	title	genres
rank			
1	0.408380	Austin Powers: The Spy Who Shagged Me (1999)	Comedy
2	0.362344	Spaceballs (1987)	Comedy Sci-Fi
3	0.358246	Billy Madison (1995)	Comedy
4	0.342534	Big Daddy (1999)	Comedy
5	0.330416	Austin Powers: International Man of Mystery (1...	Comedy
6	0.314861	Star Wars: Episode I - The Phantom Menace (1999)	Action Adventure Fantasy Sci-Fi
7	0.294553	Mission: Impossible 2 (2000)	Action Thriller
8	0.265733	Me, Myself and Irene (2000)	Comedy
9	0.262665	Matrix, The (1999)	Action Sci-Fi Thriller
10	0.260806	GoldenEye (1995)	Action Adventure Thriller

2. 基于电影风格的可视化

统计不同职业的评分记录数量

每种职业的评分记录数量：

```
{'K-12 student': 23290,
 'self-employed': 46021,
 'scientist': 22951,
 'executive/managerial': 105425,
 'writer': 60397,
 'homemaker': 11345,
 'academic/educator': 85351,
 'programmer': 57214,
 'technician/engineer': 72816,
 'other or not specified': 130499,
 'clerical/admin': 31623,
 ...
 'artist': 50068,
 'tradesman/craftsman': 12086,
 'customer service': 21850,
 'retired': 13754,
 'doctor/health care': 37205}
```

统计每个电影的风格

movie_id	title	genres	Horror	Musical	Action	Drama	Comedy	Film-Noir	Animation	...	Crime	Western	Adventure	Sci Fi
0	1	Toy Story (1995)	Animation Children's Comedy	0	0	0	0	1	0	1	...	0	0	0
1	2	Jumanji (1995)	Adventure Children's Fantasy	0	0	0	0	0	0	0	...	0	0	1
2	3	Grumpier Old Men (1995)	Comedy Romance	0	0	0	0	1	0	0	...	0	0	0
3	4	Waiting to Exhale (1995)	Comedy Drama	0	0	0	1	1	0	0	...	0	0	0
4	5	Father of the Bride Part II (1995)	Comedy	0	0	0	0	1	0	0	...	0	0	0

收集每个风格的均值，分风格将数据归一化；同时收集每个genre中各个职业的评分数据

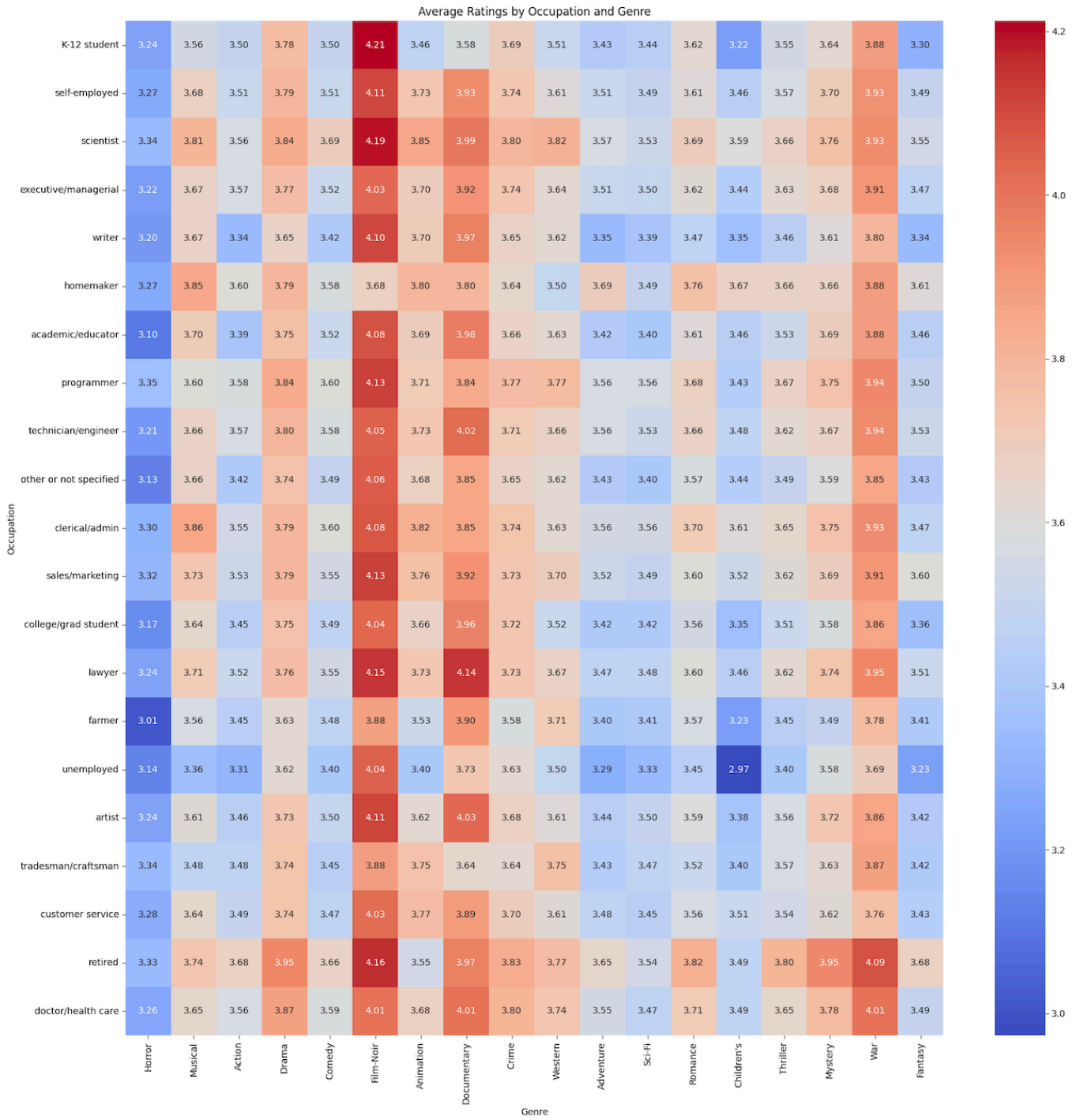
注： `portion` 是指观看某一职业观看某一类电影（评分）的人数与该职业评分记录数量之比。注意到每一职业的 `portion` 之和均比1大，这是由于一个电影可以同时属于多个类，对一个电影的一个评分会在不同类别的 `count` 中被计算多次

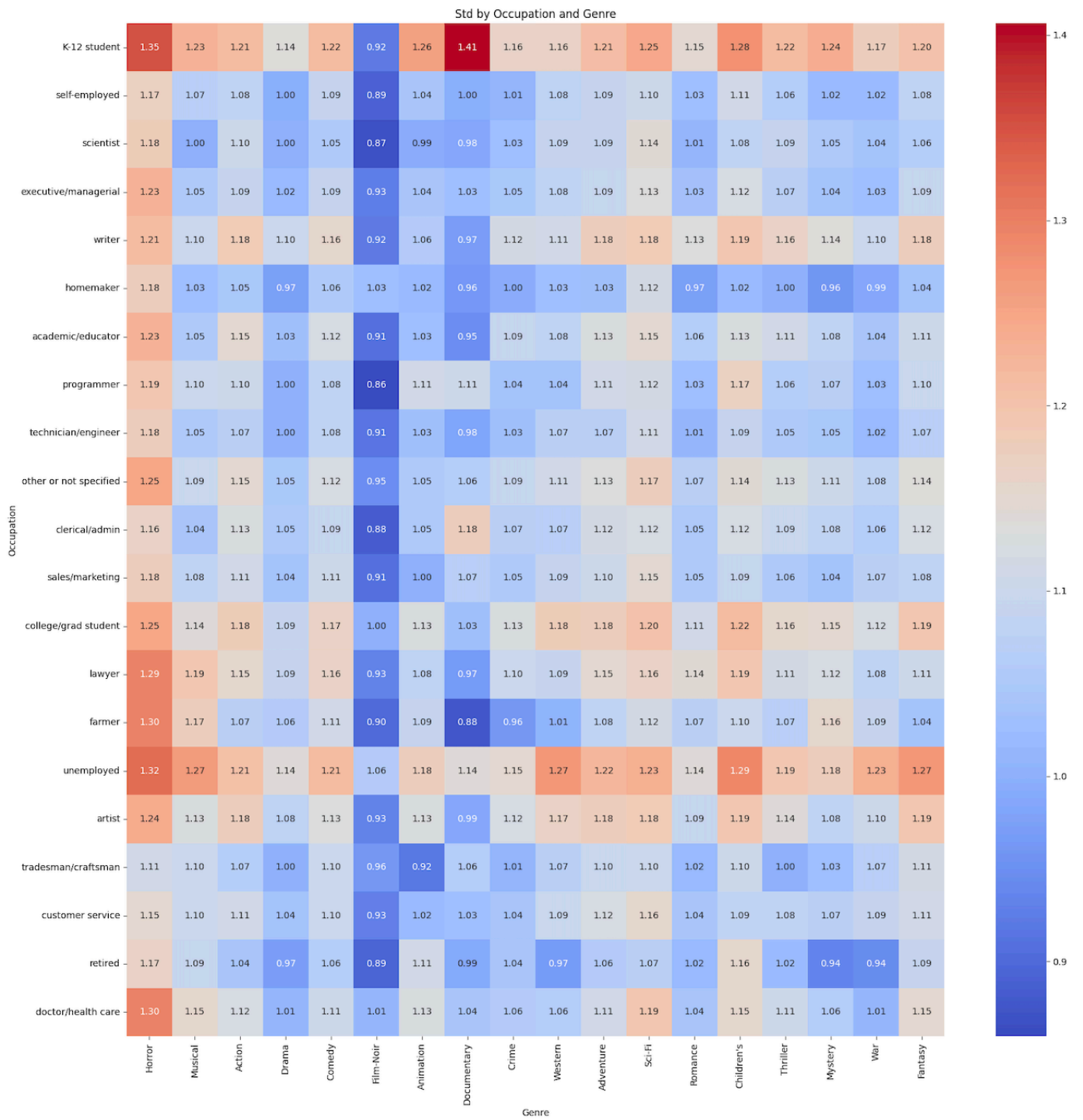
```
df_2.loc[genre, (occupation, 'portion')] =
    df_2.loc[genre, (occupation, 'count')] / occupation_count[occupation]
```

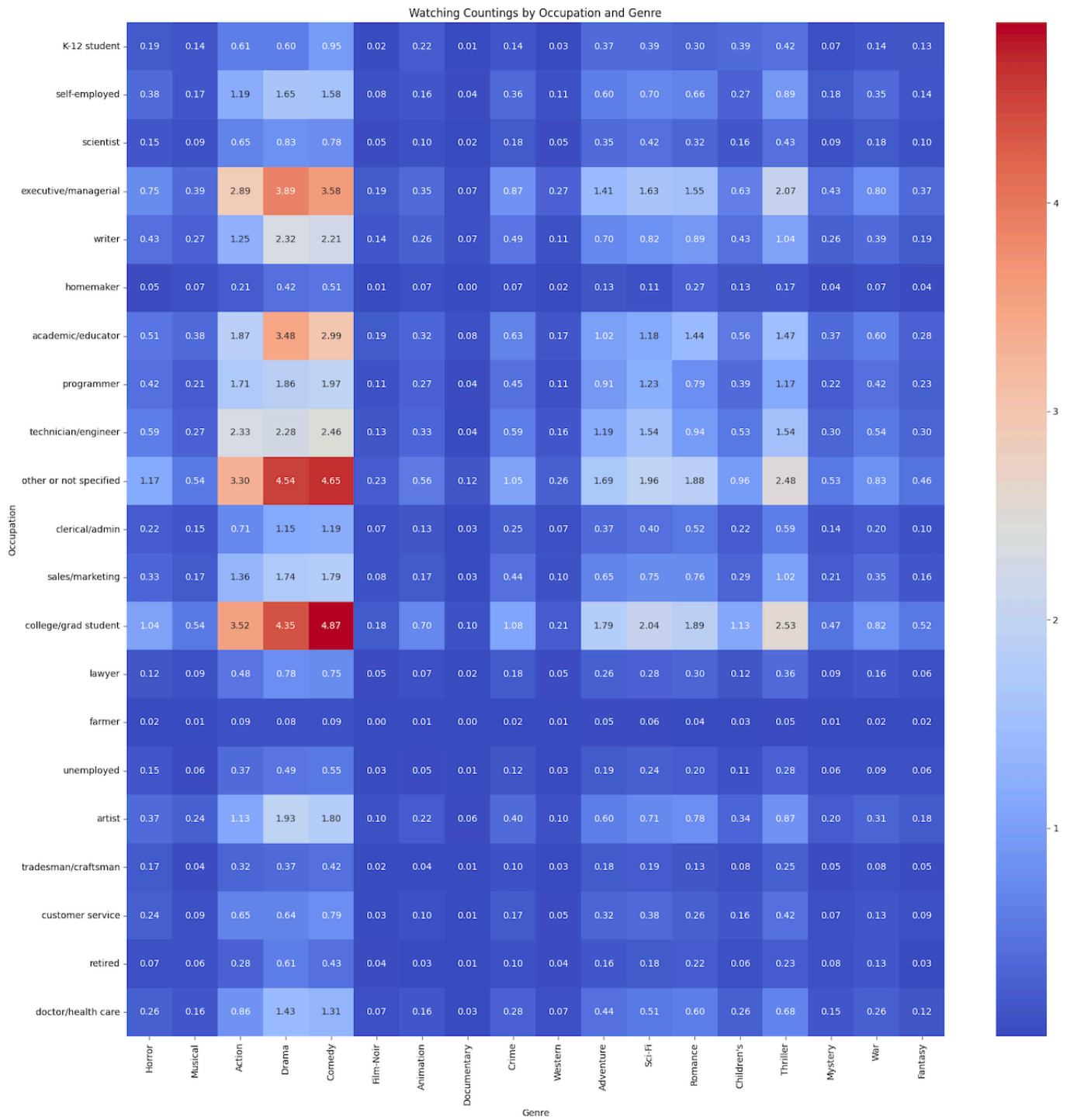
	K-12 student				self-employed				scientist		...	customer service		retired
	mean	std	count	portion	mean	std	count	portion	mean	std	...	count	portion	mean
Horror	3.237795	1.352573	1905.0	0.081795	3.266436	1.172127	3757.0	0.081637	3.336032	1.180146	...	2390.0	0.109382	3.328234
Musical	3.556738	1.228350	1410.0	0.060541	3.678006	1.069264	1705.0	0.037048	3.805987	0.998916	...	888.0	0.040641	3.736486
Action	3.497116	1.214723	6067.0	0.260498	3.510326	1.083843	11863.0	0.257774	3.559685	1.104162	...	6493.0	0.297162	3.679048
Drama	3.782167	1.135164	6000.0	0.257621	3.791983	0.997404	16489.0	0.358293	3.836070	1.000104	...	6369.0	0.291487	3.949047
Comedy	3.497200	1.215747	9465.0	0.406398	3.507570	1.087338	15787.0	0.343039	3.687170	1.054164	...	7908.0	0.361922	3.663825
Film-Noir	4.212766	0.923024	235.0	0.010090	4.105583	0.891307	824.0	0.017905	4.190476	0.867887	...	329.0	0.015057	4.161058
Animation	3.463956	1.257100	2164.0	0.092915	3.729696	1.043754	1613.0	0.035049	3.847805	0.992283	...	974.0	0.044577	3.545732
Documentary	3.581633	1.406344	98.0	0.004208	3.933492	0.997781	421.0	0.009148	3.987500	0.984071	...	97.0	0.004439	3.970588
Crime	3.687085	1.159803	1355.0	0.058179	3.737569	1.007939	3620.0	0.078660	3.801012	1.025424	...	1734.0	0.079359	3.832487
Western	3.513333	1.163760	300.0	0.012881	3.606457	1.076754	1146.0	0.024902	3.815029	1.087169	...	546.0	0.024989	3.767816
Adventure	3.425658	1.209904	3726.0	0.159983	3.513258	1.088450	6034.0	0.131114	3.567662	1.090267	...	3228.0	0.147735	3.650803
Sci-Fi	3.443795	1.245555	3932.0	0.168828	3.493593	1.103095	7024.0	0.152626	3.529564	1.142481	...	3789.0	0.173410	3.544457
Romance	3.624415	1.146542	2990.0	0.128381	3.612211	1.028827	6617.0	0.143782	3.689911	1.010342	...	2649.0	0.121236	3.824095
Children's	3.220679	1.277001	3888.0	0.166939	3.457459	1.110650	2715.0	0.058995	3.592405	1.081688	...	1597.0	0.073089	3.485849
Thriller	3.554131	1.215418	4212.0	0.180850	3.571828	1.061087	8945.0	0.194368	3.663175	1.086226	...	4238.0	0.193959	3.802019
Mystery	3.636612	1.236328	732.0	0.031430	3.702542	1.019931	1849.0	0.040177	3.756608	1.050652	...	718.0	0.032860	3.946292
War	3.880144	1.174791	1385.0	0.059468	3.926107	1.023735	3478.0	0.075574	3.933114	1.037381	...	1348.0	0.061693	4.085757
Fantasy	3.298039	1.202021	1275.0	0.054745	3.494278	1.082646	1398.0	0.030377	3.545362	1.062934	...	934.0	0.042746	3.678019

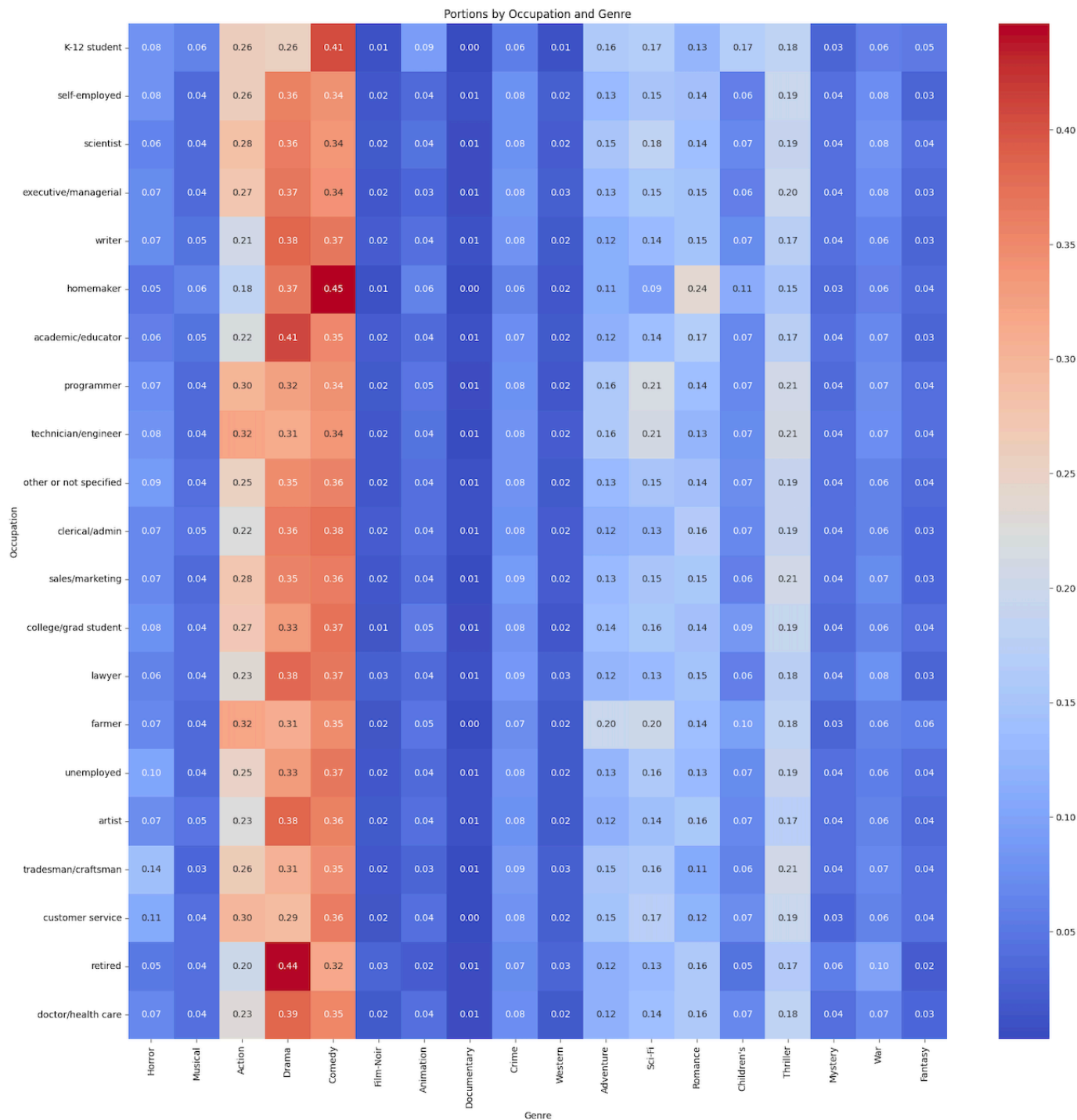
数据可视化

这里以热图形式呈现数据，更多的可视化图像在文件夹 `./figures` 中





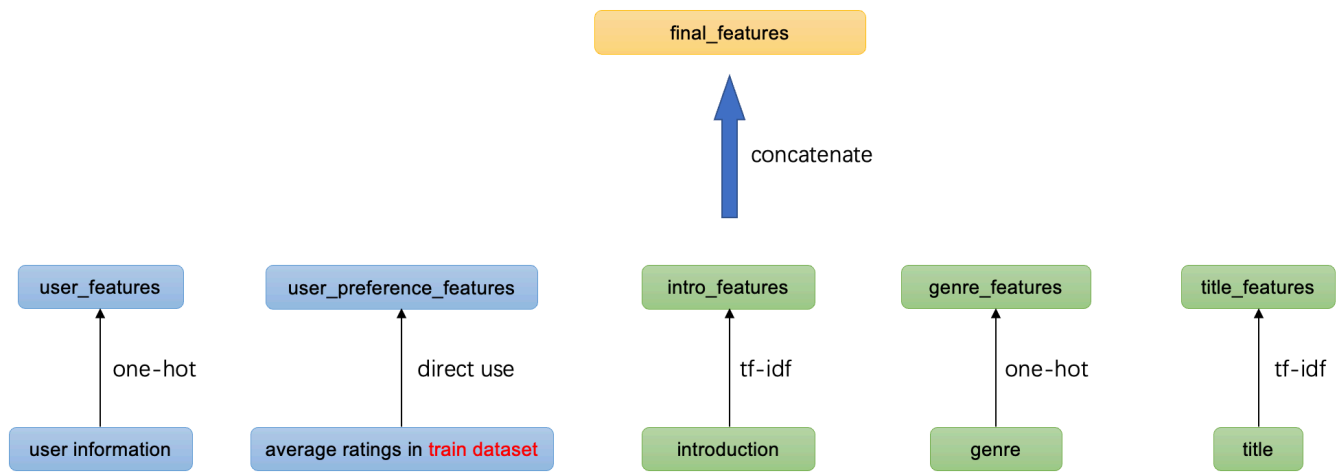




二、用户评分预测

1. 特征工程：特征的选择与构建

对每一个<用户，电影>对构建特征，如下图所示：



有三点需要注意：

- 1) 整个过程中没有对任何特征进行降维，测试表明降维会使后续模型预测效果显著变差，这可能是因为在特征向量中融合了one-hot编码得到的类别特征，对这些特征降维似乎是不合理的。
- 2) 加入了训练集用户的评分数据作为特征，这是合理的，因为我们并没有使用测试集的有关数据。使用该数据作为特征可以在一定程度上提升模型预测效果。
- 3) 构建特征的过程中对数据进行了一些处理，这里展现的只是大致过程，具体过程与得到的特征模式参见代码。这里展示一部分提取训练集特征的过程：

```

1 # 求训练集中每一种职业对每一类电影的平均评分
2 column = []
3 for occupation in occupations:
4     column.append(occupation)
5 avg_rating_record = pd.DataFrame(np.zeros((len(genre_list), len(column))), index=genre_list, columns=[column])
6
7 for occupation in occupations:
8     train_occupation = train[train['occ_desc'] == occupation]
9     # display(train_occupation.head())
10    for genre in genre_list:
11        tmp = train_occupation[train_occupation[genre]==1]
12        avg_rating_record.loc[genre, occupation] = tmp['rating'].mean()
13 avg_rating_record = avg_rating_record.T
14 print('avg_rating_record:')
15 avg_rating_record.head()

```

✓ 0.9s Python

avg_rating_record:

	Horror	Musical	Action	Drama	Comedy	Film-Noir	Animation	Documentary	Crime	Western	Adventure	Sci-Fi
K-12 student	3.252027	3.579710	3.500000	3.801501	3.502276	4.217877	3.477472	3.842857	3.695735	3.571429	3.437500	3.44971
self-employed	3.287132	3.699193	3.506300	3.804676	3.513744	4.101208	3.707355	4.006192	3.740728	3.620614	3.504893	3.49287
scientist	3.347009	3.784203	3.559869	3.843655	3.687173	4.191549	3.812958	4.040650	3.808571	3.819951	3.573818	3.53948
executive/managerial	3.244391	3.681097	3.567344	3.772326	3.524235	4.021132	3.719356	3.940154	3.731245	3.619205	3.508833	3.50554
writer	3.198243	3.675991	3.334652	3.670800	3.426184	4.120000	3.689622	4.014679	3.648732	3.647715	3.346957	3.38448

```
1 # 根据训练集的avg_rating_record构建训练集的用户偏好特征
2 train_preference_features = []
3 for i in range(len(train)):
4     train_preference_features.append(avg_rating_record.loc[train.iloc[i]['occ_desc'], :].values)
5 train_preference_features = np.array(train_preference_features).reshape(len(train), -1)
```

✓ 1m 4.7s Python

```
1 train_preference_features.shape
```

✓ 0.0s Python

(787156, 18)

```
1 # 构建用户信息的one-hot特征
2 encoder = OneHotEncoder(sparse_output=False)
3 train_user_features = encoder.fit_transform(train[['gender', 'age_desc', 'occ_desc']])
4 train_user_features = pd.DataFrame(train_user_features, columns=encoder.get_feature_names_out(['gender', 'age_desc', 'occ_desc']))
5 train_user_features.head()
```

✓ 0.2s Python

	gender_F	gender_M	age_desc_18-24	age_desc_25-34	age_desc_35-44	age_desc_45-49	age_desc_50-55	age_desc_56+	age_desc_Under 18	occ_desc_K-12 student
0	0.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	0.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0
2	1.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0
3	0.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	0.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0

5 rows x 30 columns

```
1 # 构建电影类别的one-hot特征
2 train_genre_features = train.iloc[:, -18:]
3 train_genre_features.head()
```

✓ 0.0s Python

	Horror	Musical	Action	Drama	Comedy	Film-Noir	Animation	Documentary	Crime	Western	Adventure	Sci-Fi	Romance	Children's	Thriller	Mys
0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	0	1	0	0	0	0	0	1	0
2	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
3	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0

```
1 # 构建电影intro的特征
2 vectorizer1 = TfidfVectorizer(stop_words='english')
3 sparse_train_intro_features= vectorizer1.fit_transform(train['intro'])
```

✓ 5.4s Python

```
1 # 构建电影title的特征
2 vectorizer2 = TfidfVectorizer(stop_words='english')
3 sparse_train_title_features= vectorizer2.fit_transform(train['name'].apply(lambda x: x[:8]))
```

✓ 1.2s Python

```
1 # 将构建的所有特征连接起来得到最终的特征向量
2 sparse_train_user_features = csr_matrix(train_user_features)
3 sparse_train_genre_features = csr_matrix(train_genre_features)
4 sparse_train_preference_features = csr_matrix(train_preference_features)
5 train_final_features = hstack([sparse_train_user_features, sparse_train_genre_features, train_preference_features, sparse_train_title_features])
```

✓ 0.9s Python

2. 模型选择与训练

经过测试，我们最终采用基于决策树的catboost模型。

关键在于使用回归模型而不是分类模型，实验发现分类模型（CatBoostClassifier）得到的mse大约比回归模型（CatBoostRegressor）的高0.24。

```
1 catboost_model = CatBoostRegressor(iterations=2000, learning_rate=0.2)
2 data_pool = Pool(train_final_features, y_train)
3 catboost_model.fit(data_pool)
```

✓ 2m 3.8s Python

0:	learn: 1.1084103	total: 143ms	remaining: 4m 46s
1:	learn: 1.1034865	total: 208ms	remaining: 3m 27s
2:	learn: 1.0982933	total: 275ms	remaining: 3m 3s
3:	learn: 1.0953170	total: 346ms	remaining: 2m 52s
4:	learn: 1.0918828	total: 419ms	remaining: 2m 47s
5:	learn: 1.0898084	total: 519ms	remaining: 2m 52s
6:	learn: 1.0879331	total: 593ms	remaining: 2m 48s
7:	learn: 1.0858072	total: 652ms	remaining: 2m 42s
8:	learn: 1.0834889	total: 715ms	remaining: 2m 38s
9:	learn: 1.0820354	total: 767ms	remaining: 2m 32s
10:	learn: 1.0805589	total: 816ms	remaining: 2m 27s
...			
1996:	learn: 0.9511101	total: 1m 54s	remaining: 173ms
1997:	learn: 0.9510978	total: 1m 54s	remaining: 115ms
1998:	learn: 0.9510858	total: 1m 55s	remaining: 57.5ms
1999:	learn: 0.9510797	total: 1m 55s	remaining: 0us

3. 预测与评估

```
1 # 评估模型
2 y_pred = catboost_model.predict(test_final_features)
3 mse = mean_squared_error(y_test, y_pred)
4 print("The mean squared error (MSE) on test set: {}".format(mse))
```

✓ 0.0s Python

The mean squared error (MSE) on test set: 0.937281555203504

最终得到的 $mse < 1$ ，满足题目要求。但是经过大量尝试与参数调整，仍然无法使 $mse < 0.8$ ，猜测需要进一步发掘数据的特征，进行更加细致的特征提取。

三、电影海报根据内容分类

1. 无监督聚类

图像特征提取

根据题目要求，从电影海报中提取颜色直方图和灰度直方图作为特征。

```
# 提取颜色直方图和灰度直方图特征，在无监督聚类中使用（题目有要求）
rgb_image = np.array(image)
gray_image = cv2.cvtColor(rgb_image, cv2.COLOR_RGB2GRAY)

# 计算彩色直方图
color_hist = cv2.calcHist([rgb_image], [0, 1, 2], None, [8, 8, 8], [0, 256, 0, 256, 0, 256])
color_hist = cv2.normalize(color_hist, color_hist).flatten()

# 计算灰度直方图
gray_hist = cv2.calcHist([gray_image], [0], None, [256], [0, 256])
gray_hist = cv2.normalize(gray_hist, gray_hist).flatten()

# 连接两个直方图并加入kmeans_features中
kmeans_features.append(np.hstack((color_hist, gray_hist)))
```

降维并进行kmeans聚类

使用PCA对数据降维，之后将海报聚类为五个类别，这五个类别应当是根据色彩和灰度信息划分的。

```
1 from sklearn.cluster import KMeans
2 from sklearn.decomposition import PCA
3
4
5 pca = PCA(n_components=50, random_state=42)
6 pca_features = pca.fit_transform(np.array(kmeans_features))
7
8 kmeans = KMeans(n_clusters=5, random_state=42)
9 df['kmeans_cluster'] = kmeans.fit_predict(pca_features)
10
11 df
```

✓ 8.4s

Python

	movie_id	features	kmeans_cluster
	0	63 [0.03181022, 0.4679307, 0.8066899, 0.4369812, ...	1
	1	823 [0.54533017, 2.6612434, 0.052591085, 0.4809450...	4
	2	3578 [0.04319999, 0.386104, 0.8370001, 0.0, 0.74675...	0
	3	2100 [0.10941267, 2.0392597, 0.9851427, 0.60722953,...	1
	4	1409 [0.17741236, 1.0916624, 0.19859573, 0.00152080...	3
	...	...	...
	2933	3213 [0.75683874, 1.126783, 4.2811904, 0.48052168, ...	3
	2934	190 [0.104476385, 1.7208096, 0.23858008, 0.5907070...	1
	2935	184 [1.6203073, 2.804204, 0.45903713, 0.41096118, ...	3
	2936	2119 [0.46478897, 0.6600336, 0.2783797, 0.24570668,...	3
	2937	1376 [0.9255244, 1.0727818, 0.2613289, 1.006686, 1....	1

2938 rows x 3 columns

结果可视化

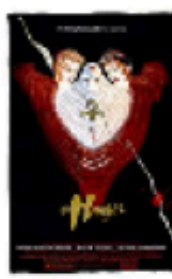
Class 0



Class 0



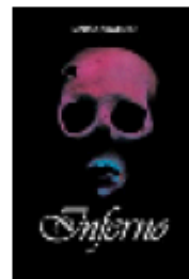
Class 0



Class 0



Class 0



Class 1



Class 1



Class 1



Class 1



Class 1



Class 2



Class 2



Class 2



Class 2



Class 2



Class 3



Class 3



Class 3



Class 3



Class 3



Class 4



Class 4



Class 4



Class 4



Class 4



2. 有监督聚类

图像特征提取

这里采用基于CNN的img2vec模型直接将图像转化为特征向量，与简单的提取颜色和灰度信息相比，该模型能够在一定程度上反映海报的内容信息，有利于我们根据海报将电影归入不同的风格类别。

其他特征的提取

我们从电影的名称和简介，以及用户对该电影的评分情况中分别提取特征信息。这里采用了BERT模型提取电影名称和简介的特征。

模型的选择、训练与测试

我们采取**集成学习**的方式搭建模型，对每一类特征我们均在**相同的训练集上**训练一个支持多分类的XGBClassifier模型，然后在相同的测试集上进行分类预测（输出属于每个类别的概率），使用投票法将各个模型的分类概率结果按照一定权重比例进行加和，设定合适的阈值以达到最佳的预测结果。具体过程如下：

```
1 # 提取特征和标签
2 image_X = merged_df['features'].values.tolist()
3 intro_X = intro_features
4 title_X = title_features
5 rating_X = rating_features
6 y = np.array(merged_df['genre_vector'].tolist())
7
8 # 划分训练集和测试集，注意要使用相同的随机种子以保证划分一致
9 image_X_train, image_X_test, image_y_train, image_y_test = train_test_split(image_X, y, test_size=0.2, random_state=42)
10 intro_X_train, intro_X_test, intro_y_train, intro_y_test = train_test_split(intro_X, y, test_size=0.2, random_state=42)
11 title_X_train, title_X_test, title_y_train, title_y_test = train_test_split(title_X, y, test_size=0.2, random_state=42)
12 rating_X_train, rating_X_test, rating_y_train, rating_y_test = train_test_split(rating_X, y, test_size=0.2, random_state=42)
```

✓ 0.0s

Python

```

1 # 根据图像特征训练分类器
2 image_model = XGBClassifier(max_depth=2)
3 image_model.fit(image_X_train, image_y_train)
4 image_y_pred = image_model.predict_proba(image_X_test)

```

✓ 4.9s

Python

```

1 # 根据电影简介训练分类器
2 intro_model = XGBClassifier(max_depth=3)
3 intro_model.fit(intro_X_train, intro_y_train)
4 intro_y_pred = intro_model.predict_proba(intro_X_test)

```

✓ 9.8s

Python

```

1 # 根据电影名称训练分类器
2 title_model = XGBClassifier(max_depth=5)
3 title_model.fit(title_X_train, title_y_train)
4 title_y_pred = title_model.predict_proba(title_X_test)

```

✓ 16.2s

Python

```

1 # 根据用户评分情况训练分类器
2 rating_model = XGBClassifier(max_depth=4)
3 rating_model.fit(rating_X_train, rating_y_train)
4 rating_y_pred = rating_model.predict_proba(rating_X_test)

```

✓ 1.0s

Python

```

1 # 对预测结果进行加权, 设定阈值得到输出
2 sum = 1.25 * intro_y_pred + 1 * image_y_pred + 0.5 * title_y_pred + 1.4 * rating_y_pred
3 maxi = 0
4 maxi_th = -1
5 for th in range(0, 5000):
6     tmp = (sum > (th / 1000)).astype(int)
7     if accuracy_score(image_y_test, tmp) > maxi:
8         maxi = accuracy_score(image_y_test, tmp)
9         maxi_th = th
10
11 y_pred = (sum > maxi_th / 1000).astype(int)

```

✓ 2.0s

Python

```

1 # 评估模型性能
2 accuracy = accuracy_score(image_y_test, y_pred)
3 print("Accuracy:", accuracy)
4 print("Classification Report:")
5 print(classification_report(image_y_test, y_pred))

```

✓ 0.0s

Python

Accuracy: 0.23809523809523808

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.03	0.07	29
1	1.00	0.08	0.15	51
2	0.58	0.58	0.58	181
3	0.67	0.04	0.08	49
4	0.00	0.00	0.00	8
5	1.00	0.06	0.12	77
6	0.64	0.80	0.71	251
...				
macro avg	0.51	0.13	0.16	1042
weighted avg	0.67	0.35	0.37	1042
samples avg	0.51	0.42	0.44	1042

最终得到的准确率在(0.2, 0.4]之间。